Math for Data Science - Lab 3

Course: Math for Data Science

This lab has two halves. The first is a tour of the **discrete distributions**: we build up the Bernoulli, binomial and hypergeometric from an urn model. (The Poisson, the fourth member of that tour, opens lab 4.) The second half is about **expectation and joint distributions**: the mean and variance of the binomial, the law of the unconscious statistician, and reading a joint PMF table for its margins, its conditionals, and whether its two variables are independent.

Exercises 1-4 are the core of the lab. Exercises 5, 6 and 7 are **optional** - the multinomial, the way conditioning on a common effect can manufacture a dependence out of nothing, and a joint distribution of your own invention to draw and pull apart. Do them if you have time, or come back to them later; nothing later in the course depends on them.

Setup

Run the cell below once to import everything we need.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from math import comb
from scipy import stats

# The lecture's joint-PMF pictures, shipped in the `m4ds/` folder beside this
notebook.
from m4ds.plotting.joint_pmf import joint_with_marginals, plot_joint_pmf
from m4ds.probability.joint_pmf import joint_marginals
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: PMF Practice

1. Suppose you have an urn filled with w white balls and b black balls. You draw n balls out of the urn *with replacement*. What is the probability that the first trial yields a white ball?

here your solution

2. Define the random variable X as the number of white balls in a *single* draw from part (1). Write down its PMF.

here your solution

3. What known distribution did you just write down, and what are its parameters?

here your solution

4. Now run six trials of the same sampling with replacement from part (1). Define the random variable Y as the number of white balls in the six trials. Write down the PMF of Y.

here your solution

5. What known distribution did you just write down, and what are its parameters?

here your solution

6. Now what if you draw n balls *without replacement*? Let Z be the number of white balls in $n = 3$ trials. Write down a PMF for Z.

here your solution

7. Note that Z follows the **hypergeometric distribution**, defined as

$$P(Z = k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$$

with n the number of trials, N the total population size (number of balls), and K the number of success states (white balls). Check that your answer to part (6) corresponds to the hypergeometric distribution.

here your solution

8. Code up a small simulation to verify that this process indeed follows the hypergeometric distribution.

a. First, create an urn with 10 white balls and 5 black balls in it, and generate 1000 iterations of 3 draws from this urn, *without* replacement. Save the number of white balls drawn in each iteration into an array.

```
# here your code
```

b. Now use NumPy's built-in hypergeometric sampler to do the same thing. Summarize the two distributions you've made and compare them.

> **Tip:** `rng.hypergeometric(ngood, nbad, nsample, size=...)` is the equivalent of R's `rhyper(nn, m, n, k)`. `pd.Series(...).describe()` gives you R's `summary()`.

```
# here your code
```

c. Now plot the two arrays as histograms overlaid on top of each other, so the comparison is visual as well as numerical. Label your axes.

```
# here your code
```

Exercise 2: Deriving the Mean and Variance of the Binomial Distribution

1. Begin by deriving the mean and variance of X_1, a Bernoulli-distributed random variable - this will help us.

here your solution

2. Now let $X = X_1 + X_2 + ... + X_n$. Thus X is the number of successes in n i.i.d. Bernoulli trials and is distributed Binomial(n, p). Use this to derive the mean and variance of X.

here your solution

3. Check both results numerically: simulate many Binomial$(n = 20, p = 0.3)$ draws and compare the sample mean and variance to np and $np(1-p)$.

> **Tip:** NumPy's `.var()` divides by n, while R's `var()` divides by $n-1$. For a sample this large the difference is invisible, but pass `ddof=1` if you want to match R exactly.

```
# here your code
```

4. The variance step above needed independence; the expectation step did not. Let's see what happens when independence fails as badly as it possibly can.

Take the draws you just made and set $Y = X$ - the same random variable, perfectly dependent on itself. Compute $\mathrm{Var}(X + Y)$ from the simulation and compare it to the two candidate answers: $\mathrm{Var}(X) + \mathrm{Var}(Y)$, and $4\,\mathrm{Var}(X)$. Which one wins, and why?

Then check that $E(X + Y) = E(X) + E(Y)$ holds anyway.

```
# here your code
```

here your solution

Exercise 3: Expectation, and Where Linearity Stops

Linearity of expectation is such a workhorse that it is tempting to assume expectation passes through *any* function. It does not. $E(g(X))$ and $g(E(X))$ are different numbers for almost every non-linear g, and the tool for computing the first one is the **law of the unconscious statistician** (LOTUS):

$$E(g(X)) = \sum_x g(x)\, P(X = x)$$

Note what LOTUS does *not* ask you to do: you never have to work out the PMF of $g(X)$. You apply g to the values of X and weight by the PMF of X.

1. Let X be the result of one roll of a fair six-sided die. Compute $E(X)$ and $E(X^3)$ by hand, directly from the definition.

here your solution

2. Confirm this by simulation. Roll a die many times, then compute three things: the average of the cubes, the cube of the average, and the LOTUS sum. Which two agree?

```
# here your code
```

here your solution

3. *[optional]* Repeat the comparison for $g(x) = \sqrt{x}$, which is concave rather than convex. Which of $E(g(X))$ and $g(E(X))$ is larger now?

```
# here your code
```

here your solution

Exercise 4: Joint, Marginal and Conditional PMFs

A joint distribution does not always arrive with a formula attached. Very often you are handed one as nothing more than a **table**. Everything you need is still in there: the margins, the conditionals, and the answer to whether the two variables are independent.

Here is a joint PMF of two random variables, $X \in \{1, 2, 3, 4\}$ and $Y \in \{1, 2, 3\}$. Row i, column j of the table holds $P(X = j, Y = i)$.

```
JOINT = np.array([
    [8, 16, 15, 9],
    [5, 12, 14, 6],
    [2,  4,  6, 3],
]) / 100

x_vals = np.array([1, 2, 3, 4])
y_vals = np.array([1, 2, 3])

print(pd.DataFrame(JOINT, index=[f"Y={y}" for y in y_vals],
                   columns=[f"X={x}" for x in x_vals]))
```

The same table as the lecture drew it: the joint PMF as a heatmap, with the marginal of X in bars below it and the marginal of Y in bars to its right. Keep the picture in view - the next three questions are about exactly these three panels.

```
fig, (ax_joint, ax_right, ax_below) = joint_with_marginals(
    JOINT, xs=x_vals, ys=y_vals, transparent=False)
plt.show()
```

1. Before computing anything: what one property must this table have in order to be a valid joint PMF? Check it in code.

```
# here your code
```

2. Compute the two **marginal** PMFs, $P(X = x)$ and $P(Y = y)$. Remember what marginalizing means: to get the PMF of X alone, you sum the joint probabilities over every value of Y,

$$P(X = x) = \sum_{y} P(X = x, Y = y)$$

Check that each margin is itself a valid PMF.

> **Tip:** `JOINT.sum(axis=0)` collapses the rows (summing over Y), `JOINT.sum(axis=1)` collapses the columns (summing over X).

```
# here your code
```

3. Now the **conditional** PMF. Compute $P(Y = y \mid X = 3)$ for every y, using

$$P(Y = y \mid X = 3) = \frac{P(X = 3, Y = y)}{P(X = 3)}$$

In table terms: pick out the column for $X = 3$ and rescale it so that it sums to 1 again. Verify that it does.

```
# here your code
```

4. *[optional]* Plot the marginal PMF of Y next to the conditional PMF of Y given $X = 3$. Does conditioning on $X = 3$ change what you believe about Y?

```
# here your code
```

here your solution

5. Are X and Y independent? Independence of two random variables means

$$P(X = x, Y = y) = P(X = x)\,P(Y = y) \qquad \text{for every pair } (x, y)$$

In other words: the joint table would have to be exactly the **outer product** of its own margins. Build that outer product, compare it to `JOINT`, and say how many cells (if any) disagree.

> **Tip:** `np.outer(p_y, p_x)` builds the table you want, oriented the same way as `JOINT`. `np.allclose` compares two arrays entry by entry.

```
# here your code
```

here your solution

6. Here is the same idea on a 2×2 table, where you can do it in your head. We sample an adult at random. Let X indicate the presence of a gene and Y indicate developing a certain disease:

	$Y = 0$	$Y = 1$
$\boldsymbol{X = 0}$	0.72	0.03
$\boldsymbol{X = 1}$	0.20	0.05

a. Find the marginal distributions of X and of Y, and name them.

b. Find the conditional distribution of Y given $X = 1$, and name it.

c. Are X and Y independent? Show the arithmetic that settles it.

here your solution

```
# here your code
```

Exercise 5: The Multinomial Distribution

[optional]

The binomial counts successes when each trial has two outcomes. The **multinomial** is what you get when each trial has more than two - which is the usual situation with survey responses, party choice, or any categorical variable.

1. A population of voters contains 40% Green Party supporters, 40% Red Party supporters, and 20% Blue Party supporters. You randomly sample 10 voters. What is the probability of getting 2 Green supporters, 5 Red supporters, and 3 Blue supporters? (Assume you are either sampling with replacement, or sampling without replacement in a population so large that sampling with replacement is a close enough approximation.)

here your solution

2. Congratulations, you've derived the PMF of the multinomial distribution. Write down the joint PMF of the random variables X_g, X_r, and X_b if they are distributed multinomial with probabilities p_g, p_r, and p_b respectively.

here your solution

3. Write down the marginal probability that your sample of 10 contains 2 Green supporters as a sum of the relevant joint probabilities.

here your solution

4. Based on your answer to question 3, consider the general case: if $\mathbf{X} \sim \text{Mult}_k(n, \mathbf{p})$, what is the marginal distribution of one element of $\mathbf{X}$, namely X_j?

here your solution

5. Now let's think about the conditional distribution. What is the conditional probability of getting 2 Green supporters and 5 Red supporters, *conditional* on there being 3 Blue supporters in the 10-person sample?

here your solution

6. Use NumPy's multinomial sampler to create the sample in question 1 above, 10,000 times. That is, draw 10,000 samples of 10 voters each from this distribution.

a. Manually compute the joint probability of 2 Green, 5 Red, and 3 Blue supporters. b. Manually compute the marginal probability of having 2 Green supporters in the sample. Compare this to a sample generated using the binomial distribution. c. Manually compute the conditional probability of having 2 Green and 5 Red supporters, given 3 Blue. Compare this to the value you get when you compute the appropriate joint over the marginal. d. Note that the values you compared in part (b) were approximately equal, while the values you compared in part (c) were exactly equal. Why is that?

Tip: `rng.multinomial(n=10, pvals=[...], size=10_000)` returns a `(10000, 3)` array - one row per sample, one column per party. R's `rmultinom` returns the transpose of this.

```
# here your code
```

here your solution

Exercise 6: When Conditioning *Creates* Dependence

[optional]

In lab 2 you met the chess opponent and the two-year driver. Both had the same shape: an unknown type Z (skill level, or good/bad driver) drove two outcomes X and Y, which were therefore *dependent* - until you conditioned on the type, at which point they became independent. In a picture, $X \leftarrow Z \rightarrow Y$: Z is a **common cause**, and conditioning on it *removes* the dependence.

This section is about the other direction, which is far less intuitive. Suppose instead that X and Y are two independent causes of a common effect Z:

$$X \rightarrow Z \leftarrow Y$$

Z here is called a **collider**. We will see that conditioning on a collider *manufactures* a dependence between two variables that were independent to begin with.

The setting: a disease X and an unrelated injury Y occur independently in the population, each with probability 0.1. A person is hospitalized ($Z = 1$) if they have either one.

1. Simulate 200,000 people. Draw X and Y independently, then define Z. Confirm from the simulation that X and Y are independent in the population as a whole, by checking that $P(X = 1 \mid Y = 1) \approx P(X = 1)$.

```
# here your code
```

here your solution

2. Now restrict attention to the hospitalized people only - that is, condition on $Z = 1$ - and compute $P(X = 1 \mid Y = 1, Z = 1)$ and $P(X = 1 \mid Y = 0, Z = 1)$. What happened?

```
# here your code
```

here your solution

3. Put the two structures side by side and state the general lesson. In lab 2's driver problem, conditioning on the type made two dependent events independent. Here, conditioning on hospitalization made two independent events dependent. What does this tell you about the relationship between independence and conditional independence?

here your solution

Exercise 7: Draw Your Own Joint Distribution

[optional]

Exercise 4 handed you a joint table and asked you to take it apart. This one asks you to *build* one - for a story of your own - and then look at it, using the same three functions the lecture slides were drawn with:

- `joint_marginals(joint)` returns the pair `(p_x, p_y)`;
- `joint_with_marginals(joint, xs=..., ys=..., transparent=False)` draws the table with both margins, and returns `fig, (ax_joint, ax_right, ax_below)`;
- `plot_joint_pmf(joint, xs=..., ys=..., transparent=False)` draws the table alone, and takes `highlight_col=` and `dim_others=` to single out one column.

1. Invent a joint distribution. Pick two discrete variables with a handful of values each - three by three is plenty - and a story that says why they should hang together. Write the table down as *relative frequencies* (how often each pair happens, in whatever units are convenient), then **normalise** it by dividing by its total, and check that the result sums to 1.

Keep the orientation of Exercise 4: row i, column j holds $P(X = x_j, Y = y_i)$.

```
# here your code
```

2. Draw it with `joint_with_marginals`. Then read the two margins straight off the panels: the bars below the table are $P(X = x)$, the bars to its right are $P(Y = y)$. Confirm what you see against `joint_marginals`, and say in a sentence what the shape of each margin means in your story.

> **Tip:** pass `transparent=False`. The default, `True`, leaves the figure background see-through - that is for slides sitting on a coloured backdrop, and in a notebook it just looks broken.

```
# here your code
```

here your solution

3. Now condition. Pick one column j of your table - one value of X - and draw the table again with `plot_joint_pmf(..., highlight_col=j, dim_others=True)`, so the column you are conditioning on is outlined and the rest fades back. Then compute the conditional PMF $P(Y = y \mid X = x_j)$ by rescaling that column, and check it sums to 1.

> **Note:** `highlight_col` is a *column index*, counting from 0 - not the value x_j itself.

```
# here your code
```

here your solution

4. Finally, build the table your two variables *would* have had if they were independent - the outer product of the margins, `np.outer(p_y, p_x)` - and plot it next to the real one. Both tables have exactly the same margins; only the inside differs. What does independence *look like* in the heatmap?

> **Tip:** `plot_joint_pmf(..., transparent=False, ax=ax)` draws into an axes you already have, so `fig, axes = plt.subplots(1, 2, figsize=(12, 3.6))` gives you the two panels side by side. Give both panels the same `vmax=` as well, or the colours will not be comparable.

```
# here your code
```

here your solution